Contents
Introduction
Join the Community
Scale Your Data by Partitioning Data
Back of the envelope Estimation
Ch1Approach for System Design Interviews
Design a Rate Limiter
Design Consistent Hashing
Design a Key-value Store
Design a Unique ID Generator
Design a Web Crawler
Design a Newsfeed System
Design a Chat System
Design a Search Typeahead System
Design a Search Autocomplete System
Design a Video Sharing System
Design a Ride-hailing System
Distributed Message Queue
Leading a distributed Relational Database System
Google Maps
Distributed Message Queue
Distributed Search System
Ch 34 Object Storage
Real time Gaming Leaderboard
Payment System
Digital Wallet
Stock Exchange
The Learning Continues

20
Distributed Message Queue

In this chapter, we explore a popular question in system design interviews: design a distributed message queue. In modern architecture, systems are broken up into small and independent building blocks that can interact with well-defined interfaces. Message queues provide communication and coordination for these building blocks. What benefits do message queues bring?

- Decoupling: message queues add a stable interface between two services so they can be updated independently.
- Improved scalability: We can scale producers and consumers independently based on traffic load. For example, during peak hours, more consumers can be added to handle the increased traffic.
- Increased availability: If part of the system goes offline, other components can still interact with the message queue.
- Better performance: Message queues make asynchronous communication easy. Producers can add messages to a queue without waiting for the response and consumers consume messages whenever they are available. This don't need to wait for each other.

Figure 1 shows some of the most popular distributed message queues on the market.

Figure 1 Popular distributed message queues

Message queues in event streaming platforms

Strictly speaking, RabbitMQ and Apache MQ are message queues as they are event streaming platforms. However, there is a distinction between message queues and event streaming platforms. Traditional message queues (MQ) are designed for event streaming platforms (Kafka, Pulsar). For example, RabbitMQ, which is a typical message queue, added an optional streaming feature to allow replayable messages. Conversely, Kafka, which is a typical event streaming platform, can be configured to function as a traditional message queue. In this chapter, we will discuss how to design a distributed message queue that can be used as a typical distributed message queue.

In this chapter, we will design a distributed message queue with **additional features, such as long data retention, repeated consumption of messages, etc.** that are typically only available in event streaming platforms. These additional features make the design more interesting and challenging. Some of the questions we will ask while we design this design could be simplified if the focus of our interviewer is traditional messaging.

Step 1 - Understand the Problem and Establish Design Scope

In a nutshell, the basic functionality of a message queue is straightforward: producers send messages to a queue, and consumers receive messages from the queue. However, there are many details that need to be clarified between the interviewer and candidate, such as message retention, data formats, etc. The following set of questions is a good start for clarifications and scope the case.

Candidate: What is the format and average size of messages? Is text only? Is multimedia allowed?
Interviewer: Text messages in serialized formats such as JSON or protocol buffers (protobuf).

Candidate: Can messages be repeatedly consumed?
Interviewer: Yes, but this should be configurable. Different by different consumers. Note that this is an added feature, a traditional distributed message queue does not usually guarantee delivery order. However, Kafka, a typical event streaming platform, is a traditional message queue that can be repeatedly consumed.

Candidate: How long should messages be kept? Do we need to support data retention?
Interviewer: Yes, let's assume data retention is two weeks. This is an added feature. A traditional distributed message queue does not usually guarantee delivery order.

Candidate: Does data need to be persisted on the disk?
Interviewer: Yes, let's assume data retention is two weeks. This is an added feature. A traditional message queue does not usually guarantee delivery order.

Candidate: How many producers and consumers are we going to support?
Interviewer: Multiple producers and consumers.

Candidate: In order delivery, consumers need to support consumer groups, consumers can move ahead, or rewind, at will, to read any offset.
Interviewer: Yes, definitely want to support at least once, ideally, we should support all of them and make them configurable.

Candidate: What is the target throughput and end-to-end latency?
Interviewer: It should support high throughput for use cases like log aggregation. It should also support low latency delivery for more traditional messaging use cases.

Well-thought-out clarifications are important for a successful design. In addition to the functional requirements, the following characteristics are also important for a distributed message queue:

- Scalability: the system should be scalable in order to handle a large volume of messages.
- Consumers consume messages from a message queue.
- Messages can be consumed repeatedly or only once.
- Messages are persisted on the disk.
- Messages are in the binary format.
- Ability to retain messages for consumers in the order they were written to the queue.
- Data delivery semantics (at least once, at most once, exactly once) can be configured by users.

Candidate: Data delivery semantics (at least once, at most once, exactly once) can be configured by users.

Non-functional requirements

- High throughput or low latency, configurable based on use cases.
- Scalable. The system should be distributed in nature. It should be able to support a sudden surge in message volume.
- Persistent and durable. Data should be persisted on the disk and replicated across multiple nodes.

Adjustments for traditional message queues

Traditional message queues like RabbitMQ do not focus on strong or eventual consistency in event streaming platforms. Traditional message queues like RabbitMQ do not focus on strong or eventual consistency in event streaming platforms. Messages are stored in a sorted order of magnitude smaller than the capacity required for event streaming platforms. Traditional message queues do not typically maintain message ordering. The messages can be consumed in a different order than they were produced. These differences provide us with the design scope for a distributed message queue.

Step 2 - Propose High-Level Design and Get Buy-In

Fully decoupled and scalable messaging system.

Figure 2 shows the key components of a message queue and the simplified interactions between these components.

Figure 2 Key components in a message queue

- Producer sends messages to a message queue.
- Consumer subscribes to a queue and consumes the subscribed messages.
- Message queue is a service in the middle that decouples the producers and the consumers, allowing each of them to operate and scale independently.
- Both producer and consumer are clients to the distributed middle, while the message queue is the server. The clients and servers communicate over the network.

Messaging models

The message queue messaging model is powerful and extremely flexible, having numerous messaging models for various use cases. In this section, we discuss the most popular messaging models and how they are used to power the message queue.

Point-to-point

This model is commonly found in traditional message queues. In a point-to-point model, a message is sent to a queue and consumed by one, and only one, consumer. The sender produces something to be consumed and the receiver consumes the message. Once, but each message can only be consumed by a single consumer. In Figure 3, message 2 is only consumed by consumer 2.

Figure 3 Point-to-point model

Once the consumer acknowledges that a message is consumed, it is removed from the queue. There is no data retention in the point-to-point model. In contrast, our design includes a persistence layer that keeps the messages for two weeks, which allows messages to be consumed repeatedly.

While our design model supports a point-to-point model, its capabilities may more directly to the publish-subscribe model.

Publish-subscribe

First, let's introduce a new concept, the topic. Topics are the categories used to organize messages. Each topic has a name that is unique across the entire message queue. Messages are sent to and read from specific topics.

In the publish-subscribe model, a message is sent to a topic and received by any consumers subscribing to the topic. As shown in Figure 4, message 2 is consumed by both consumer 1 and consumer 2.

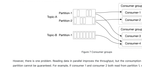


Figure 4 Publish-subscribe model

Our distributed message queue supports both models. The point-to-point model is implemented by **topics**, and the point-to-point model can be simulated by the concept of the **consumer group**, which will be introduced in the consumer group section.

Topics, partitions, and brokers

As mentioned earlier, messages are persisted by topics. Where it fits data volume in a topic may be too large to a single server to handle?

One approach is to have a topic support a **partition (shard)** concept. In this case, we can distribute a topic's data across partitions and deliver messages evenly across partitions. Think of a partition as a small subset of the messages for a topic. Partitions are evenly distributed across servers in a cluster. Each server is responsible for handling its own share of the partitions. Figure 5 shows how messages are evenly sorted among partitions for a topic. With proper hashing algorithm, we can ensure that each partition is responsible for a fair share of the workload.

Figure 5 Partitions

Each topic partition operates in the form of a queue with the FIFO (first, in, first out) mechanism. This means we can keep the order of messages inside a partition. The position of a message in the partition is called the offset.

When a message is sent to a partition, it is actually sent to one of the partitions for the topic. Each message has an optional message key and value. If a message has a key, then the message will be sent to a partition based on the key. For example, if all messages related to the message is consumed by one of the consumers.

When a consumer subscribes to a topic, it gets data from one or more of these partitions. When there are multiple consumers subscribing to a topic, each consumer is responsible for a subset of the partitions for the topic. The consumers form a **consumer group**.

The message queue cluster with brokers and partitions is represented in Figure 6.

Figure 6 Message queue cluster

Consumer group

As mentioned earlier, we need to support both point-to-point and publish-subscribe models. A **consumer group** is a set of consumers, working together to consume messages from topics.

Consumers can be organized into groups. Each consumer group can subscribe to multiple topics and maintain its own consuming offsets. Consumers in a group can be distributed across multiple servers or VM instances.

The instances in the same group consume partitions in a parallel. as in Figure 7.

- Consumer group 1 subscribes to topic A.
- Consumer group 2 subscribes to topic A and topic B.
- Topic A is subscribed by both consumer groups 1 and 2, which means the same message is consumed by multiple consumers. The partition supports the same message/event consumed.

Figure 7 Consumer group

Having more than one partition. Reading data in parallel improves the throughput but the communication cost of messages in the same partition cannot be guaranteed. For example, if consumer 1 and consumer 2 both read from partition 1, we will not be able to guarantee the message consumption order in partition 1.

The good news is we can fix this by adding a constraint, that a single partition can only be consumed by one consumer in the same consumer group. Consumers can be slow, and if a slow consumer is responsible for consuming multiple partitions, it will slow down the entire group. We rule. For example, in Figure 7, consumer 3 in group 2 can consume messages from topic B because it is consumed by Consumer 4 in the same consumer group, which is a different group.

Consumer groups are a powerful concept that allows some consumer groups, their messages in the same partition are consumed only one consumer in the same group, and other groups can read the same partition. Since a partition is the smallest unit, we can allocate enough partitions in advance to avoid the need to dynamically increase the number of partitions. To handle huge scale, we just need to add consumers.

High-Level architecture

Figure 8 shows the updated high-level design.

Figure 8 High level design

Clients:

- Producer partition messages to specific topics.
- Consumer group subscribes to topics and consumes messages.

Core service and storage:

- Broker: holds multiple partitions. A partition holds a subset of messages for a topic.
  - Storage:
    - Data: stores messages on persistent in data storage in partitions.
    - Metadata: stores configuration and properties of topics.
    - Metadata stores configuration and properties of topics and consumer's metadata storage.
- Coordination services:
  - Service discovery: which brokers are alive.
  - Leader election: one of the brokers is selected as the active controller. There is only active controller in the cluster. The controller is responsible for managing partitions.
  - Apache Zookeeper is one of popular coordination service contender.

Step 3 - Design Deep Dive

To achieve high throughput while satisfying the high data retention requirement, we made three important design choices, which we will discuss in this section:

- We chose an on-disk data structure that takes advantage of the great sequential access performance of rotational disks and the aggregated disk caching strategy of modern operating systems.
- We designed the message data structure to be backward and forward compatible, to allow for future changes to the message schema, with no disruptions. This minimizes the need for copying which is very expensive at high volume and high traffic scenarios.
- We designed our system to batch batching. Small I/O is an inefficient operation, both over the network and persistence storage. Batching. The producers send messages in batches. The message queue persists messages in even larger batches. The consumers fetch messages in batches when they read.

Data access patterns are a critical part of any data intensive application, because it determines how data is stored, retrieved, and managed efficiently. The way data is organized and accessed directly impacts the system's performance, scalability, and resource utilization. This section explores two key data access patterns: the message queue model and the publish-subscribe model, which are fundamental to distributed messaging systems. We will examine how these patterns influence the design and implementation of a distributed message queue, including their impact on data storage, retrieval, and processing.

Data storage

Data storage is a critical component of any distributed messaging system, and the choice of data structure can significantly impact performance and scalability. In this section, we explore three popular data structures: write-ahead log (WAL), commit log, and segmented log. We will discuss their characteristics, advantages, and how they are used in real-world systems, such as Kafka and other messaging systems.

Let's consider a hypothetical scenario where we need to store a sequence of messages for a single topic. Each message has a unique identifier and contains some data. The primary operations we need to support are writing new messages to the end of the log and reading messages from a specific offset.

Write-ahead log (WAL) is a simple, yet effective, data structure for persisting messages. In our case, we can implement a WAL as a single file to which new messages are appended. Each message is assigned a unique offset, which is a sequential number that indicates the position of the message in the file. The broker maintains an in-memory index to track the byte offsets of the last messages for each partition, enabling efficient retrieval of messages based on their offsets.

Figure 9 Reading buffer

As shown in Figure 9, the producer tries to send messages to partition 2 of topic A.

- The producer sends messages to the reading buffer.
- The reading buffer sends the replica distribution plan* from the metadata storage and batches messages based on it. Kafka uses the Nagle's algorithm to batch messages.
- The reading buffer sends batches to the appropriate brokers. Each broker persists the batch to its local disk.
- When "enough" replicas have received the batch, the leader broker persists the message. Acks determine what "enough" means. Acks can be consumed. Time is expressed in milliseconds.

*Kafka's Nagle's algorithm

The reading buffer is designed into the producer and is a buffer component is added to the producer. Both can be installed in the producer as part of the producer client library. This change brings several benefits:

- Reduces the number of network requests.
- Producers can have their own logic to determine which partition the message should be sent to.
- Batching is a major strategy to improve and avoid large batches in a single request. This increases throughput.

The choice of the message data structure is a critical component in optimizing the performance of distributed messaging systems. While the simplicity of write-ahead logs (WAL) and commit logs offers high write throughput, the ability to quickly locate specific messages becomes a crucial consideration. The segmented log approach addresses this challenge by dividing the log into smaller, manageable segments, each with its own index. This approach significantly improves the efficiency of message retrieval, especially when dealing with large volumes of data. The segmented log approach also facilitates efficient data management and cleanup, as entire segments can be easily deleted or archived when they reach a certain age or size threshold.

Figure 10 The choice of the batch size

Consumer flow

The consumer specifies its offset in a partition and receives back a chunk of events beginning from that position. An example is shown in Figure 11.

Figure 11 Consumer flow

Push vs pull

An important question to answer is whether brokers should push data to consumers, or if consumers should pull data from the brokers.

Push model

- Low latency. The broker can push messages to the consumer immediately upon receiving them.

Cons

- If the rate of consumption falls below the rate of production, consumers could be overwhelmed.
- It is difficult to deal with consumers with different processing power because the brokers control the rate at which data is transferred.

Pull model

Pros

- Consumers control the consumption rate. We can have one set of consumers process messages in real-time and another set of consumers process messages in batch mode.
- Consumers can decide to pull messages in batches. In production, we can scale out the consumers, or simply catch up when it can.

- The pull model is suitable for batch processing. In the publish-subscribe model, if a consumer is in a backlogged of whether consumers will be able to process messages immediately. If the broker sends one message at a time to the consumer and the consumer is backed up, the message will be stuck in the network or buffer. The broker can't send more messages to the consumer because it cannot process the backlog. Kafka's consumer's poll model can fill up its buffer to aggressively batching of data.

Cons

- Often time is wasted on idle. The broker may keep sending back empty responses. To overcome this issue, many message queues support long polling mode, which allows pulls to wait a specified amount of time for new messages [5].

Based on these considerations, most message queues support the pull model.

Figure 12 shows the workflow of the consumer pull model.

Figure 75 Kafka broker

1. A new consumer wants to join group 1 and subscribes to topic A. It finds the corresponding broker leader by hashing the group name. By doing so, it ensures that the same group coordinator is the same broker, which is also the coordinator of the consumer group. Despite the same broker, the coordinator group coordinator is different from the coordination service mentioned in Figure 74. The coordinator in Figure 74 is responsible for cluster metadata, while the group coordinator is responsible for consumer groups.
2. The coordinator notifies the consumer that current the group and assigns partitions to the new consumer. Different partition assignment strategies including round robin, range, etc. [1].
3. Consumer consumes messages from assigned partitions. Offsets are managed by the state storage.
4. Consumer periodically heartbeats and commits the offset to the broker. The offset is also persisted and offset controlling affects the message delivery semantics, which will be discussed shortly.

Consumer rebalancing

Consumer rebalancing describes which consumer is responsible for which subset of partitions. The process could occur when a consumer joins, when a consumer leaves, when consumer subscribes to other partitions, etc.

When consumer rebalancing occurs, the coordinator plays an important role. Let's first take a look at what a coordinator is. The coordinator is one of the brokers responsible for communication with consumers to achieve consumer rebalancing. The coordinator receives heartbeat from consumers and manages their offset on the partitions.

Let's use an example to understand how the coordinator and the consumer work together.

Figure 76 Coordinator of consumer group

Let's simulate a new consumer scenario. Assume there are 3 consumers in the group, and 4 partitions in the subscribed topic. Figure 80 shows the first step of consumer group.

Figure 77 Consumer rebalance

Figure 78 New consumer join

1. Initially, only consumer 1 is in the group. It consumes all the partitions and sends the heartbeat with the coordinator.
2. Consumer 2 wishes a request to join the group.
3. Coordinator receives the request and decides to re-balance all the consumers in the group in a passive way. When it's heartbeat is responded by the coordinator, it asks to re-join the group.

4. Since all the consumers have rejoined the group, the coordinator chooses one of them as the leader and informs all the consumers about the partition assignment.
5. The leader calculates the partition assignment plan and sends it to the coordinator. However, consumers ask the coordinator about the partition assignment plan.
6. Consumer start consuming messages from newly assigned partitions.

Figure 79 Re-balance process

Figure 80 Existing consumer leaves

1. Consumer 3 and 2 are in the same consumer group.
2. Consumer 3 wants to the re-start share, so it requests to leave the group.
3. The coordinator knows the re-balance. Since the coordinator knows the consumer 3 is the coordinator, it asks 2 to re-join the group.
4. The remaining stand are the same as the state shown in Figure 76.

Figure 81 Describes the process of existing consumer in a cluster.

Figure 82 Existing consumer leaves

1. Consumer 3 and 2 have heartbeats with the coordinator.
2. Consumer 3 wishes to re-start share, so it sends a request to leave the group.
3. The coordinator knows the re-balance. Since the coordinator is not the consumer 3 to be coordinator. Since the coordinator doesn't get any heartbeat response within a specified amount of time, it assumes that consumer 3 is dead. The consumer is dead.
4. The coordinator triggers the rebalance process.
5. The following steps are the same as the steps in the previous scenario.

Note that we finished the detour on consumer and consumer flows, let's come back and finish the deep dive on the rest of the message queue broker.

State storage

In the message queue broker, the state storage stores:

- The mapping between partitions and consumers.
- The last committed offset for consumer groups for each partition. As shown in Figure 25, the last consumed offset for consumer group 1 in 3 and the offset for consumer group 2 is 10.

Figure 83 Last consumed offset of consumer groups

For example, as shown in Figure 25, a consumer in group 1 consumer messages from the partition in sequence and commits the consumed offset 3. The status of the message before and at offset 3 are already consumed. If the consumer crashes, another new consumer in the same group will resume consumption by reading the last consumed offset from the state storage.

The data access patterns for consumer states are:

- Frequent read and write operations but the volume is not high.
- Data consistency is critical to avoid stale reads.
- Random read and write operations.
- Data consistency is important.

Lots of storage solutions can be used for storing the consumer state data. Considering the data consistency and fast read/write requirements, a KV store like Zookeeper is a good choice. Kafka has moved the offset storage from Zookeeper to Kafka brokers. Interested readers can read the reasons behind the move [2].

Metadata storage

The metadata storage stores the configuration and properties of topics, including a number of partitions, retention period, and distribution of replicas.

Metadata is small in data and frequently and the data volume is small, but it has a high consistency requirement. Zookeeper is a good choice for storing metadata.

ZooKeeper

By leading previous sections, you probably have already learned that Zookeeper is very helpful for designing a distributed message queue system. Zookeeper is a popular, high-performance, and high-availability open-source distributed coordination service. It is commonly used in distributed computing for service synchronization, service discovery, and leading-heights [3].

Zookeeper is used to simplify our design as shown in Figure 23.

Figure 23. Zookeeper

Let's briefly go over the change.

- Metadata will state storage are moved to Zookeeper.
- Zookeeper stores the state storage and the last offset for messages.
- Zookeeper helps with the leader election of the broker cluster.

Replication

In distributed systems, hardware issues are common and cannot be ignored. Disks go bad when a disk is damaged or fails permanently. Data in the disk is inaccessible and data is lost.

So in Figure 24, we add replication for Leaders, distributed copies of the broker cluster.

For each partition, the highlighted replicas are the leaders and the others are followers. Followers only send messages to the leader replicas. The follower replicas pulling new messages from the leader. Once messages are successfully replicated to the leader, the leader returns an acknowledgment to the producer. We will go into detail about how to define "enough" in the in-sync Replication section below.

Figure 24 Replication

The distribution of replicas for each partition is called a replica distribution plan. For example, the replica distribution plan in Figure 24 can be described as:

- Partition 1 of topic A: 3 replicas, leader is broker 1, followers is broker 2 and 3.
- Partition 2 of topic A: 3 replicas, leader is broker 2, followers is broker 1 and 3.
- Partition 3 of topic A: 3 replicas, leader is broker 3, followers is broker 1 and 2.

Who makes the replica distribution plan? It works as follows: when a broker joins the coordination service, one of the broker nodes is elected as the leader. It generates the replica distribution plan and persists the plan in metadata storage. All the brokers now can work according to the plan.

If you are interested in knowing more about replications, check out Chapter 5 "Replication" of the book "Design Data-intensive Applications" [4].

In-sync replicas

We mentioned that messages are persisted in multiple partitions to avoid single node failure, and each partition has multiple replicas. Messages are only written to the leader, and followers synchronize the data from the leader. Data replication can lead to many issues in system.

In sync replicas (ISR) refer to replicas that are "in-sync" with the leader. The definition of "in-sync" depends on the topic configuration. For example, if the value of replica.lag.max.messages is 4, it means that as long as the number of behind the leader by no more than 3 messages, the replica is in-sync. Let's take a look at Figure 25 for an example.

Let's use an example as shown in Figure 25 to show how ISR works.

Figure 25 Replication

Figure 25 illustrates the case with broker 3 down. Two new messages are written to the leader, but not committed yet. Committed offset means that all replicas have received the offset and already persisted successfully in all the replicas.

- Replica 2 lags behind Kafka lags behind the leader, so they are in ISR and can meet lag messages.
- Replica 3 lags too much, so it will be kicked within the configuring lag time, and it is out of ISR. When it catches up again, it can be added to ISR.

Figure 26 Kafka ISR works.

Why do we need ISR? The reason is that ISR reflects the trade-off between performance and durability. If producers don't want to lose any messages, the safest way is to let the leader wait for the replicas to send acknowledgments. But a slow replica will cause the entire system to become slow or unavailable.

Now that we've discussed ISR, let's take a look at acknowledgment settings. Producers can choose to receive acknowledgments until the message has been written to the replica, leader, or ISR.

Acks=all

Figure 26 illustrates the case with broker 3 down. Acks=all, the producer will wait after all ISRs have received the message. This means it takes a longer time to send a message because we have to wait for the slowest ISR. Acks=1 gives the strongest message durability.

Figure 26 acks=all

Acks=1

With Acks=1, the producer receives an ACK once the leader persists the message. The latency is improved to not waiting for slow replicas. However, if the message has not been replicated to other replicas before the leader crashes, the message is lost. In follower nodes, then the message is lost. This setting is suitable for low latency scenarios where occasional data loss is acceptable.

Figure 27 acks=1

Acks=0

The producer keeps sending messages to the leader without waiting for any acknowledgment, and it never retries. This method provides the lowest latency of the use of performance but is unsure if the data. The setting might be good for use cases like collecting metrics or logging data since data loss is high and occasional data loss is acceptable.

Figure 27 acks=0

Configuring ACK allows us to trade durability for performance.

Now let's look at the consumer side. The easiest way is for the consumer connect to a leader replica to consume messages.

You might be wondering if the leader replica would be overwhelmed by this design and why messages are not read from ISRs. The reasons are:

- Design and operational simplicity.
- Performance: Data replication will dispatched to only one consumer within a consumer group, this limits the number of connections in the leader replica.
- The number of connections to the leader replicas is usually not large as long as a topic is not super hot.

In a nutshell, the leader replica is not overwhelmed by the number of partitions and consumers.

In some scenarios, reading from the leader replica might not the best option. For example, if a consumer is located in a different data center from the leader replica, the read will be across data centers. This will be very slow and costly. In this case, we can let the broker stores ISRs, consumer clusters can choose use the reference replicas and read from the closest one.

ISR is very important. How does it determine if a replica is ISR or not? Usually, the leader for every partition tracks the ISR list by monitoring if every replica fetch itself in a reasonable amount of time. Slow or unresponsive replicas are considered as not in-sync. Let's look at the implementations in reference implementation [5] [6].

Scalability

By now we have made great progress designing the distributed message queue system. In the next step, let's evaluate the scalability of different system components.

Producer

The producer is conceptually much simpler than the consumer because it doesn't need group coordination. The scalability of producers can easily be achieved by adding or removing producer instances.

Consumer

Consumer groups are isolated from each other, so it is easy to add or remove a consumer group. Inside a consumer group, the scalability of consumers is mainly handled by the broker because a consumer group is consumed by a broker in a cluster. With consumer groups and the reference mechanism, the scalability and fault tolerance of consumers can be achieved.

Broker

Before discussing scalability on the broker side, let's first consider the failure recovery of brokers.

Figure 28 Broker crash crashes

Let's use an example in Figure 28 to explain how failure recovery works.

1. Assume there are 3 brokers and the partition replica distribution plan is shown below:
   1. Partition 1 of topic A: replica is broker 1 (leader), 2, and 3
   2. Partition 2 of topic A: replica is broker 2 (leader), 1, and 3
   3. Partition 3 of topic A: replica is broker 3 (leader), 1, and 2
2. Broker 3 crashes. Broker 1 and broker 2 detect the broker crash. The partition distribution plan is changed to:
   1. Partition 1 of topic A: leader is broker 1 (leader) and 2
   2. Partition 2 of topic A: leader is broker 2 (leader) and 1
   3. Partition 3 of topic A: leader is broker 1 and 2
3. The broker controller detects broker 3 is down and generates a new partition distribution plan for the remaining broker nodes:
   1. Partition 1 of topic A: leader is broker 1 (leader) and 2
   2. Partition 2 of topic A: leader is broker 2 (leader) and 1
   3. Partition 3 of topic A: leader is broker 1 (leader), 1, and 2 (new)
4. The new replica broker 1 starts to catch up with the leader.

To make the broker fault tolerant, have two additional considerations:

1. The minimum broker size. Kafka/Pulsar first ensure replicas the minimum much receive before a message is considered to be successfully committed. The higher the number, the safer. But on the other hand, we need to balance latency and safety.
2. The number of replicas. The more replicas, the more disk space required. Usually, the number is set to 3 in the code. It is also a source of resources to evaluate before it is changed. Therefore replicas should not be too large.

If all the replicas of a partition crash, the data for that partition is lost forever. When changing the number of replicas and replica locations, Kafka provides a command line tool, kafka-reassign-partitions.sh, to automatically. It's safe to distribute replicas across data centers but will incur cross-data center traffic, so it's a non-trivial data replication. Kafka is, unfortunately, data hoarding can lead to copy and paste the data across data centers, which is a non-trivial data replication.

Now let's get back to discussing the scalability of brokers. The recommended approach for scaling brokers is to add new broker nodes and rebalance or removed.

However, there is a better approach. The broker controller can temporarily allow more replicas in the system than the number of replicas in the config file. When the newly added broker catches up, we can remove the ones that are no longer needed. Let's use an example as shown in Figure 29 to understand the approach.

Broker 1
Broker 2
Broker 3
Broker 1
Broker 2
Broker 3
Broker 4
Broker 1
Broker 2
Broker 3
Broker 4
Broker 1
Broker 2
Broker 3
Broker 4

- Producer sends a message synchronously or asynchronously with a response callback, setting ack=1 or ack=all, to make sure messages are delivered to the broker. If the message delivery fails or timeouts, the producer will keep retrying.
- Consumer fetches the message and commits the offset only after the data is successfully processed. If the consumer fails to process the message, it will re-consume the message so there won't be data loss. On the other hand, if a consumer processes the message but fails to commit the offset to the broker, the message will be re-consumed when the consumer restarts, resulting in duplicates.
- A message might be delivered more than once to the brokers and consumers.

Figure 33 At-least once

Use cases: With at-least once, messages won't be lost but the same message might be delivered multiple times. While not ideal from a user perspective, at-least once delivery semantics are usually good enough for use cases where data duplication is not a big issue or deduplication is possible on the consumer side. For example, with a unique key in each message, a message can be rejected when writing duplicate data to the database.

Exactly once

Exactly once is the most difficult delivery semantic to implement. It is friendly to users, but it has a high cost for the system's performance and complexity.

Figure 34 Exactly once

Use cases: Financial-related use cases (payment, trading, accounting, etc.). Exactly once is especially important when duplication is not acceptable and the downstream service or third party doesn't support idempotency.

Advanced features

In this section, we talk briefly about some advanced features, such as message filtering, delayed messages, and scheduled messages.

Message filtering

A topic is a logical abstraction that contains messages of the same type. However, some consumer groups may only want to consume messages of certain subtypes. For example, the ordering system sends all the activities about the order to a topic, but the payment system only cares about messages related to checkout and refund.

One option is to build a dedicated topic for the payment system and another topic for the ordering system. This method is simple, but it might raise some concerns.

- What if other systems ask for different subtypes of messages? Do we need to build dedicated topics for every single consumer request?
- It is a waste of resources to save the same messages on different topics.
- The producer needs to change every time a new consumer requirement comes, as the producer and consumer are now tightly coupled.

Therefore, we need to resolve this requirement using a different approach. Luckily, message filtering comes to the rescue.

A naive solution for message filtering is that the consumer fetches the full set of messages and filters out unnecessary messages during processing time. This approach is flexible but introduces unnecessary traffic that will affect system performance.

A better solution is to filter messages on the broker side so that consumers will only get messages they care about. Implementing this requires some careful consideration. If data filtering requires data decryption or deserialization, it will degrade the performance of the brokers. Additionally, if messages contain sensitive data, they should not be readable in the message queue.

Therefore, the filtering logic in the broker should not extract the message payload. It is better to put data used for filtering into the metadata of a message, which can be efficiently read by the broker. For example, we can attach a tag to each message. With a message tag, a broker can filter messages in that dimension. If more tags are attached, the messages can be filtered in multiple dimensions. Therefore, a list of tags can support most of the filtering requirements. To support more complex logic such as mathematical formulae, the broker will need a grammar parser or a script executor, which might be too heavyweight for the message queue.

With tags attached to each message, a consumer can subscribe to messages based on the specified tag, as shown in Figure 35. Interested readers can refer to the reference material [15].

Figure 35 Message filtering by tags

Delayed messages & Scheduled messages

Sometimes you want to delay the delivery of messages to a consumer for a specified period of time. For example, an order should be closed if not paid within 30 minutes after the order is created. A delayed verification message (check if the payment is completed) is sent immediately but is delivered to the consumer 30 minutes later. When the consumer receives the message, it checks the payment status. If the payment is not completed, the order will be closed. Otherwise, the message will be ignored.

Different from sending instant messages, we can send delayed messages to temporary storage on the broker side instead of to the topics immediately, and then deliver them to the topics when time's up. The high-level design for this is shown in Figure 36.

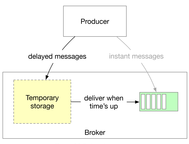


Figure 36 Delayed messages

Core components of the system include the temporary storage and the timing function.

- The temporary storage can be one or more special message topics.
- The timing function is out of scope, but here are 2 popular solutions:
  - Dedicated delay queues with predefined delay levels [16]. For example, RocketMQ doesn't support delayed messages with arbitrary time precision, but delayed messages with specific levels are supported. Message delay levels are 1s, 5s, 10s, 30s, 1m, 2m, 3m, 4m, 6m, 8m, 9m, 10m, 20m, 30m, 1h, and 2h.
  - Hierarchical time wheel [17].

A scheduled message means a message should be delivered to the consumer at the scheduled time. The overall design is very similar to delayed messages.

Step 4 - Wrap Up

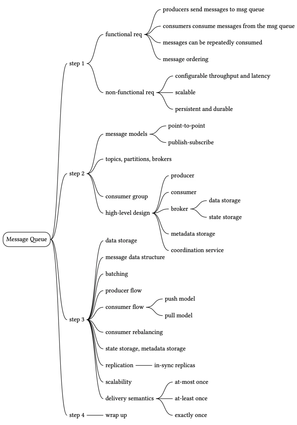
In this chapter, we have presented the design of a distributed message queue with some advanced features commonly found in data streaming platforms. If there is extra time at the end of the interview, here are some additional talking points:

- Protocol: it defines rules, syntax, and APIs on how to exchange information and transfer data between different nodes. In a distributed message queue, the protocol should be able to:
  - Cover all the activities such as production, consumption, heartbeat, etc.
  - Effectively transport data with large volumes.
  - Verify the integrity and correctness of the data.

  Some popular protocols include Advanced Message Queuing Protocol (AMQP) [18] and Kafka protocol [19].
- Retry consumption. If some messages cannot be consumed successfully, we need to retry the operation. In order not to block incoming messages, how can we retry the operation after a certain time period? One idea is to send failed messages to a dedicated retry topic, so they can be consumed later.
- Historical data archive. Assume there is a time-based or capacity-based log retention mechanism. If a consumer needs to replay some historical messages that are already truncated, how can we do it? One possible solution is to use storage systems with large capacities, such as HDFS or object storage, to store historical data.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter summary



Reference Materials

[1] Queue Length Limit: https://www.rabbitmq.com/docs/maxlength

[2] Apache ZooKeeper - Wikipedia: https://en.wikipedia.org/wiki/Apache_ZooKeeper

[3] etcd: https://etcd.io/

[4] Comparison of disk and memory performance: https://deliveryimages.acm.org/10.1145/1570000/1563874/jacobs3.jpg

[5] Cyclic redundancy check: https://en.wikipedia.org/wiki/Cyclic_redundancy_check

[6] Push vs. pull: https://kafka.apache.org/documentation/#design_pull

[7] Kafka 2.0 Documentation: https://kafka.apache.org/20/documentation.html#consumerconfigs

[8] Kafka No Longer Requires ZooKeeper: https://towardsdatascience.com/kafka-no-longer-requires-zookeeper-ebfbf3862104

[9] Martin Kleppmann. (2017). 'Replication' in *Designing Data-Intensive Applications*. O'Reilly Media. pp. 151-197

[10] ISR in Apache Kafka: https://www.cloudkarafka.com/docs/dictionary.html

[11] Apache Kafka allow consumers fetch from closest replica: https://cwiki.apache.org/confluence/display/KAFKA/KIP-392%3A+Allow+consumers+to+fetch+from+closest+replica

[12] Hands-free Kafka Replication: https://www.confluent.io/blog/hands-free-kafka-replication-a-lesson-in-operational-simplicity/

[13] Kafka high watermark: https://ronsavinblog.wordpress.com/2016/07/29/kafka-high-watermark/

[14] Kafka mirroring: https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=27846330

[15] Message filtering in RocketMQ: https://partners-intl.aliyun.com/help/doc-detail/29543.htm

[16] Scheduled messages and delayed messages in Apache RocketMQ: https://partners-intl.aliyun.com/help/doc-detail/43349.htm

[17] Hashed and hierarchical timing wheels: http://www.cs.columbia.edu/~nahum/w6998/papers/sosp87-timing-wheels.pdf

[18] Advanced Message Queuing Protocol: https://en.wikipedia.org/wiki/Advanced_Message_Queuing_Protocol

[19] Kafka protocol guide: https://kafka.apache.org/protocol

Footnotes

1. The distribution of replicas for each partition is called a replica distribution plan ↩

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.